西北大学

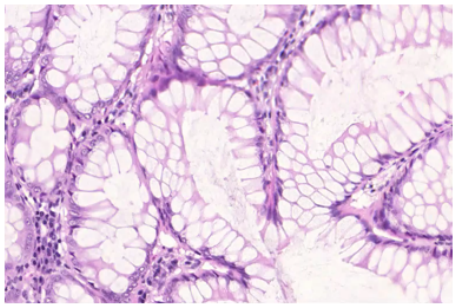
智能信息系统综合实践

实验报告

题 目:	**第九次作业**
年 级:	*2020*
专 业:	软件工程
姓 名:	*刘航*

一、 题目（原题目）

腺体组织语义分割





提供数据集
训练集：85张图像
测试集：partA 60张图像, partB 20 张图像

代码：
提供UNet代码（包含数据预处理、训练、测试、指标计算），pytorch

目标：
请在测试集中达到更好的结果
Dice, IOU等

	Precision	Recall	F1 Score	Dice Score	Hausdorff Distance
testA	0.93 (0.06)	0.93 (0.06)	0.93 (0.06)	0.93 (0.07)	83.35 (66.77)
testB	0.90 (0.08)	0.90 (0.08)	0.90 (0.08)	0.91 (0.07)	62.36 (33.02)

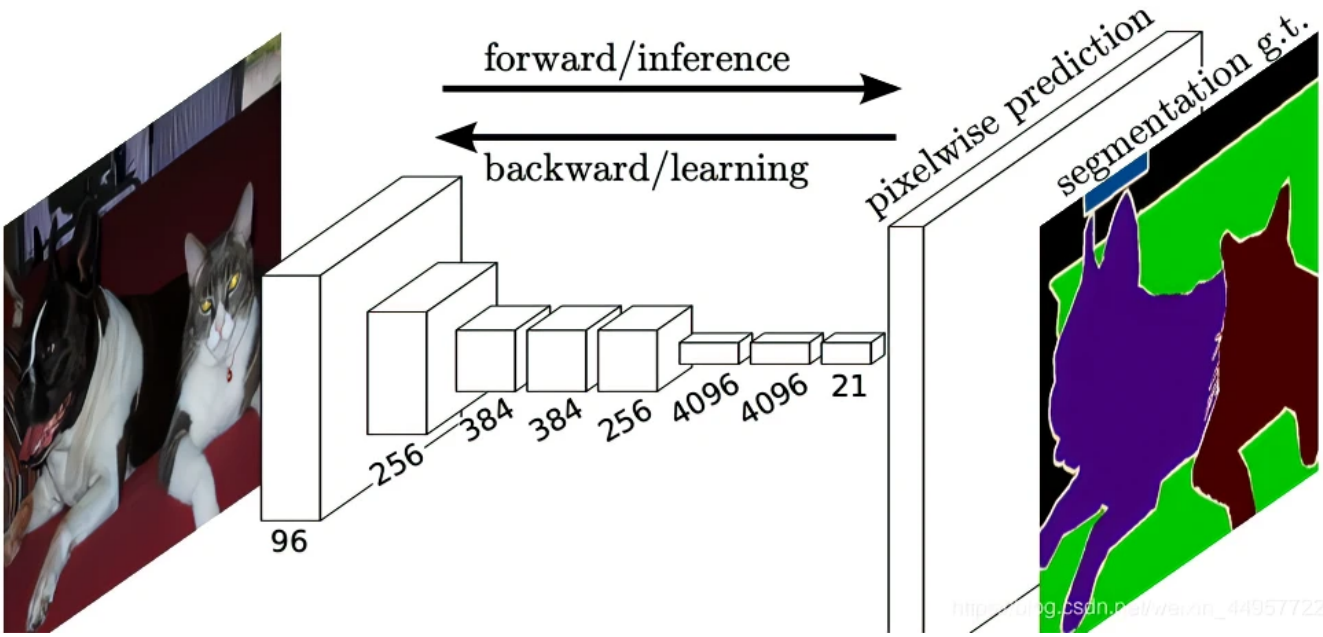
二、 解题步骤（思路 + 代码）

语义分割的任务：语义分割是一种典型的计算机视觉问题，其涉及将一些原始数据（例如，平面图像）作为输入并将它们转换为具有突出显示的感兴趣区域的掩模。

卷积神经网络分类和图像分割的区别：

CNN一般用作图像级的分类,而FCN可以对图像进行像素级的分类，从而解决了语义级别的图像分割（semantic segmentation）问题。与经典的CNN在卷积层之后使用全连接层得到固定长度的特征向量进行分类（全联接层＋softmax输出）不同，FCN可以接受任意尺寸的输入图像，采用反卷积层对最后一个卷积层的feature map进行上采样, 使它恢复到输入图像相同的尺寸，从而可以对每个像素都产生了一个预测, 同时保留了原始输入图像中的空间信息, 最后在上采样的特征图上进行逐像素分类(最后逐个像素计算softmax分类的损失, 相当于每一个像素对应一个训练样本).

如下图所示



简单的来说，FCN与CNN的区域在把于CNN最后的全连接层换成卷积层，输入一张原始图,输出的是一张已经Label好的图片,做到了端到端.

图像分割中使用的损失函数：

交叉熵Loss

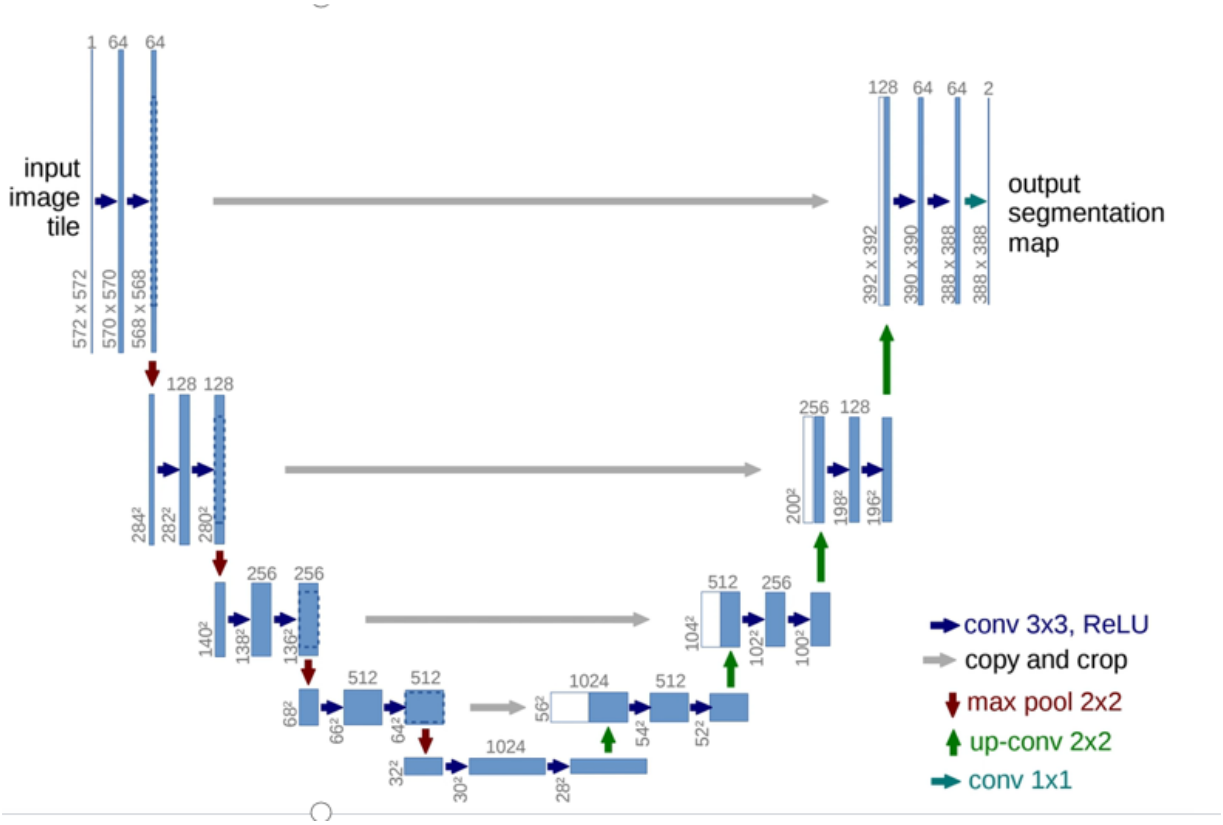
$$L = -\sum_{c=1}^{M} y_c log(p_c)$$

其中，M表示类别数，y_c是一个 one-hot 向量，元素只有0和1两种取值，如果该类别和样本的类别相同就取1，否则取0，至于P_c表示预测样本属于c的概率。

当类别数M等于2的时候，这个损失就是二元交叉熵$Loss$，在Pytorch中提供了一个单独的实现。

Unet网络介绍：

一个U型网络结构，2015年在[图像分割]领域大放异彩，unet被大量应用在分割领域。它是在FCN的基础上构建，它的U型结构解决了FCN无法上下文的信息和位置信息的弊端。



Unet网络非常的简单，前半部分就是特征提取，后半部分是上采样。在一些文献中把这种结构叫做**编码器-解码器结构**

如上图，Unet 网络结构是对称的，形似英文字母 U 所以被称为 Unet。整张图都是由蓝/白色框与各种颜色的箭头组成，其中，**蓝/白色框表示 feature map；蓝色箭头表示 3x3 卷积，用于特征提取；灰色箭头表示 skip-connection，用于特征融合；红色箭头表示池化 pooling，**

用于降低维度；绿色箭头表示上采样 upsample，用于恢复维度；青色箭头表示 1x1 卷积，用于输出结果。

Encoder 由卷积操作和下采样操作组成。

feature map 经过 Decoder 恢复原始分辨率，该过程除了卷积比较关键的步骤就是 upsampling 与 skip-connection。

Upsampling 上采样常用的方式有两种：1.FCN **中介绍的反卷积**；2. **插值。**这里介绍文中使用的插值方式。在插值实现方式中，bilinear 双线性插值的综合表现较好也较为常见 。

Batch Size从小到大的变化对网络影响

1、没有Batch Size，梯度准确，只适用于小样本数据库
2、Batch Size=1，梯度变来变去，非常不准确，网络很难收敛。
3、Batch Size增大，梯度变准确，
4、Batch Size增大，梯度已经非常准确，再增加Batch Size也没有用

注意：Batch Size增大了，要到达相同的准确度，必须要增大epoch。

实验过程：

数据初步分析：

查看数据分布，初步理解数据。

原始数据格式：

```
df = pd.read_csv('Grade.csv')
print(df.head())
```

```
      name  patient ID  grade (GlaS)  grade (Sirinukunwattana et al. 2015)
0   testA_1           4        benign                             adenomatous
1  testA_10          10        benign                                 healthy
2  testA_11           9        benign                                 healthy
3  testA_12          11     malignant                    poorly differentiated
4  testA_13           7     malignant                moderately differentiated
```

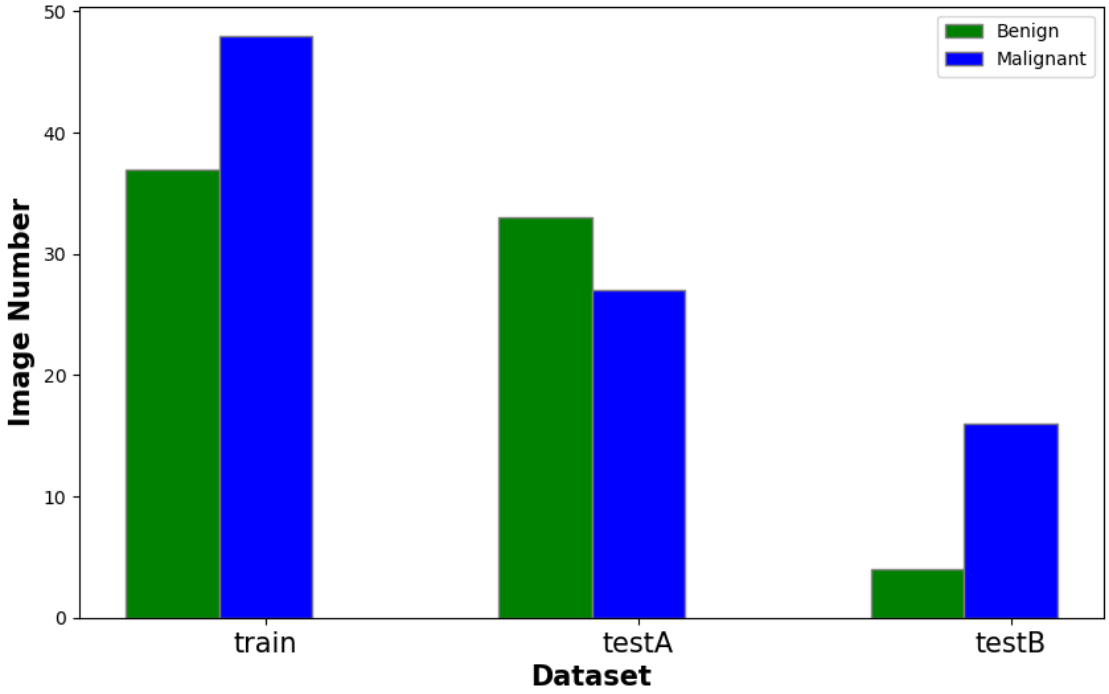
数据中有三类分别是train、testA，testB，期中train作为训练数据，testA作为验证数据集，testB作

为测试数据集。

给每个图片赋予一个类别属性：

- 'train' image => fold 0
- 'testA' image => fold 1
- 'testB' image => fold 2

查看图片的分布：



Train image, benign: 37, malignant: 48

TestA image, benign: 33, malignant: 27

TestB image, benign: 4, malignant: 16

像素数据特征的分布情况。

```
Image height:  [430 433 442 453 522]
Count:  [  1   3   2   8 151]


count    165.000000
mean     515.509091
std       21.550173
min      430.000000
25%      522.000000
50%      522.000000
75%      522.000000
max      522.000000
Name: image_height, dtype: float64
```

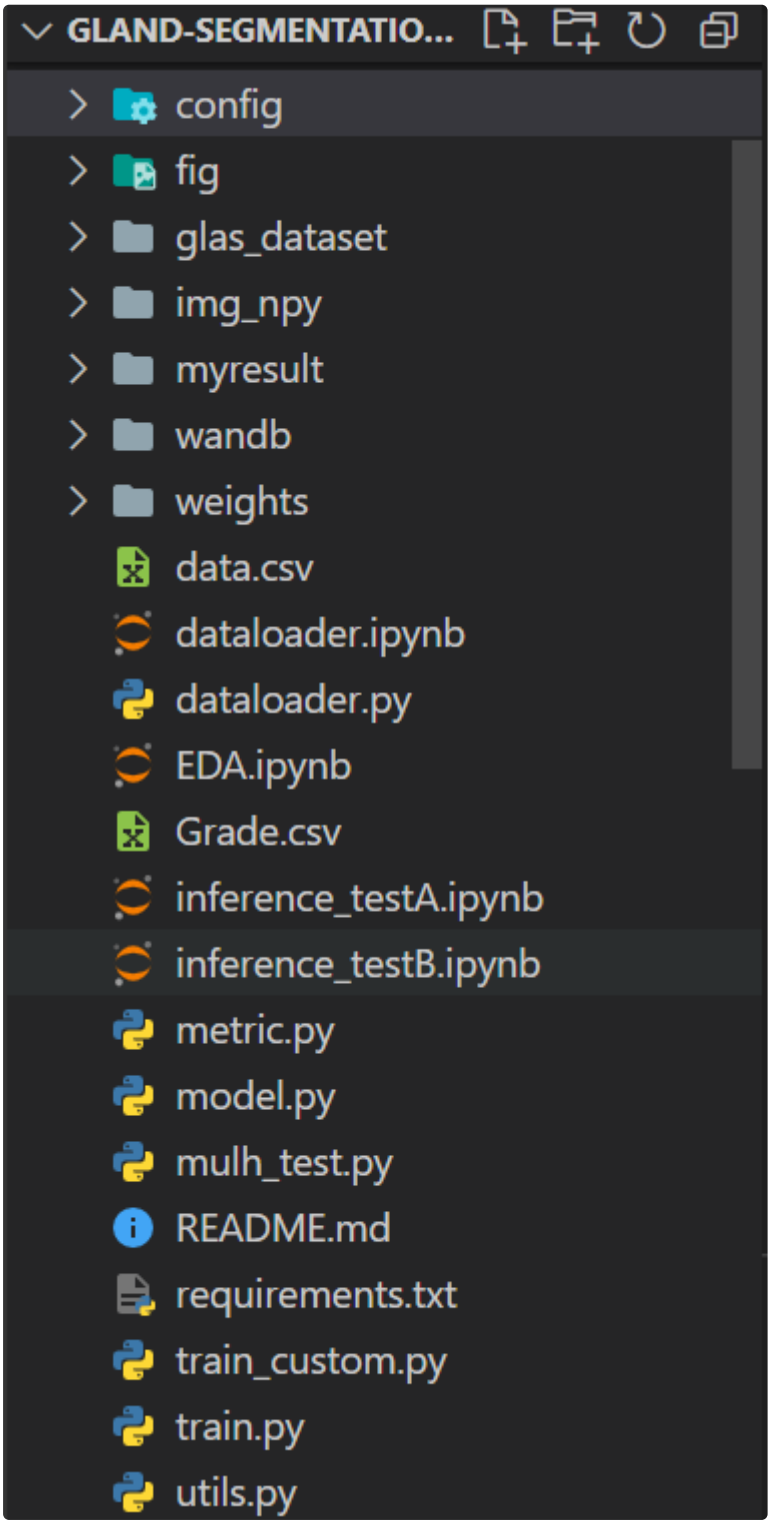
```python
# intuitively resize to size with power of 2 for easier downsampling
new_size = (512, 512)
# save directory
save_dir = 'img_npy/'
img_save_path_list = []
anno_save_path_list = []
if not os.path.exists(save_dir):
    os.makedirs(save_dir)
for name in df['name']:
    gland_img = cv2.imread(img_dir+name+'.bmp').astype('float32')
    # resize image
    gland_img = cv2.resize(gland_img, new_size)
    gland_img_save_path = save_dir+name+'.npy'
    np.save(gland_img_save_path, gland_img)
    img_save_path_list.append(gland_img_save_path)
    # ----------------------------------------
    gland_img_anno = cv2.imread(img_dir+name+'_anno.bmp').astype('float32')
    # resize anno
    gland_img_anno = cv2.resize(gland_img_anno, new_size)
    gland_img_anno_save_path = save_dir+name+'_anno.npy'
    np.save(gland_img_anno_save_path, gland_img_anno)
```

数据预处理：

将bmp格式的图片数据转换为npy格式的数据，这样可以加快读写速度。

```
# intuitively resize to size with power of 2 for easier downsampling
new_size = (512, 512)
# save directory
save_dir = 'img_npy/'
img_save_path_list = []
anno_save_path_list = []
if not os.path.exists(save_dir):
    os.makedirs(save_dir)
for name in df['name']:
    gland_img = cv2.imread(img_dir+name+'.bmp').astype('float32')
    # resize image
    gland_img = cv2.resize(gland_img, new_size)
    gland_img_save_path = save_dir+name+'.npy'
    np.save(gland_img_save_path, gland_img)
    img_save_path_list.append(gland_img_save_path)
    # ----------------------------------------
    gland_img_anno = cv2.imread(img_dir+name+'_anno.bmp').astype('float32')
    # resize anno
    gland_img_anno = cv2.resize(gland_img_anno, new_size)
    gland_img_anno_save_path = save_dir+name+'_anno.npy'
    np.save(gland_img_anno_save_path, gland_img_anno)
```

项目的结构解析:



config：里面存储的配置文件。

cvs是数据存放的文件名，和train.py放在同一个目录下。

配置的是模型的训练epoch，训练的batch_size，学习率，使用的优化器和编码模型，以及数据增强中

的每个操作被应用的概率。

fig：里面存放的是图片。

img_npy：存放的训练数据和测试数据，这个数据是npy格式的数据，这可以加快读写速度。

myresult：存放的是自己训练时候模型保存的文件。

超参数的配置：

```yaml
csv:
  value: 'data.csv'
n_class:
  value: 1
batch_size:
  value: 1
epochs:
  value: 200
learning_rate:
  value: 1e-3
optimizer:
  value: torch.optim.Adam
encoder_model:
  value: 'resnet18'
patch_size:
  value: (512, 512)
aug_prob:
  value: 0.5
```

data.cvs文件内容，这里面存储的是数据的访问路径。和标记的存放路径

fold这里表示的是这个文件是trian、testA，testB是属于那一类的。

A	B	C	D	E	F	G	H	I
name	patient_ID	grade (GlaS)	grade (Sirinukunwattana et al.	fold	image_height	image_width	img_npy	anno_npy
testA_1	4	benign	adenomatous	1	522	775	img_npy/testA_1.npy	img_npy/testA_1_anno.npy
testA_10	10	benign	healthy	1	522	775	img_npy/testA_10.npy	img_npy/testA_10_anno.npy
testA_11	9	benign	healthy	1	522	775	img_npy/testA_11.npy	img_npy/testA_11_anno.npy
testA_12	11	malignant	poorly differentiated	1	522	775	img_npy/testA_12.npy	img_npy/testA_12_anno.npy
testA_13	7	malignant	moderately differentiated	1	522	775	img_npy/testA_13.npy	img_npy/testA_13_anno.npy
testA_14	7	malignant	moderately differentiated	1	522	775	img_npy/testA_14.npy	img_npy/testA_14_anno.npy
testA_15	7	malignant	moderately differentiated	1	522	775	img_npy/testA_15.npy	img_npy/testA_15_anno.npy
testA_16	4	malignant	moderately differentiated	1	442	581	img_npy/testA_16.npy	img_npy/testA_16_anno.npy
testA_17	7	malignant	moderately differentiated	1	522	775	img_npy/testA_17.npy	img_npy/testA_17_anno.npy
testA_18	9	benign	healthy	1	522	775	img_npy/testA_18.npy	img_npy/testA_18_anno.npy
testA_19	11	benign	adenomatous	1	522	775	img_npy/testA_19.npy	img_npy/testA_19_anno.npy
testA_2	3	benign	healthy	1	453	589	img_npy/testA_2.npy	img_npy/testA_2_anno.npy
testA_20	9	benign	healthy	1	522	775	img_npy/testA_20.npy	img_npy/testA_20_anno.npy
testA_21	9	benign	healthy	1	522	775	img_npy/testA_21.npy	img_npy/testA_21_anno.npy
testA_22	11	malignant	moderately-to-poorly differenta	1	522	775	img_npy/testA_22.npy	img_npy/testA_22_anno.npy
testA_23	4	malignant	poorly differentiated	1	522	775	img_npy/testA_23.npy	img_npy/testA_23_anno.npy
testA_24	7	malignant	moderately differentiated	1	522	775	img_npy/testA_24.npy	img_npy/testA_24_anno.npy
testA_25	10	benign	healthy	1	522	775	img_npy/testA_25.npy	img_npy/testA_25_anno.npy
testA_26	7	malignant	moderately differentiated	1	522	775	img_npy/testA_26.npy	img_npy/testA_26_anno.npy
testA_27	11	benign	adenomatous	1	522	775	img_npy/testA_27.npy	img_npy/testA_27_anno.npy
testA_28	10	benign	healthy	1	522	775	img_npy/testA_28.npy	img_npy/testA_28_anno.npy
testA_29	11	malignant	moderately-to-poorly differenta	1	522	775	img_npy/testA_29.npy	img_npy/testA_29_anno.npy
testA_3	4	malignant	moderately differentiated	1	442	581	img_npy/testA_3.npy	img_npy/testA_3_anno.npy
testA_30	4	benign	adenomatous	1	522	775	img_npy/testA_30.npy	img_npy/testA_30_anno.npy
testA_31	9	benign	healthy	1	522	775	img_npy/testA_31.npy	img_npy/testA_31_anno.npy
testA_32	11	malignant	moderately-to-poorly differenta	1	522	775	img_npy/testA_32.npy	img_npy/testA_32_anno.npy
testA_33	9	benign	healthy	1	522	775	img_npy/testA_33.npy	img_npy/testA_33_anno.npy
testA_34	4	malignant	poorly differentiated	1	522	775	img_npy/testA_34.npy	img_npy/testA_34_anno.npy
testA_35	15	benign	healthy	1	453	589	img_npy/testA_35.npy	img_npy/testA_35_anno.npy

下面这行代码是在使用 `wandb`（Weights & Biases）这个工具来进行实验记录和可视化。`wandb.init()`

函数用于初始化一个新的 `wandb` 项目。其中 `project` 参数表示该项目的名称，`config` 参数表示该项

目使用的配置文件路径。

```
17
18    warnings.filterwarnings('ignore')
19    # wandb.init(project='Gland_Seg', entity='glaseg', config='config/config.yaml')
20    wandb.init(project='Gland_Seg',  config='config/config.yaml')
21    print('999')
22
23    config = wandb.config
24    print('zhis is a config', config)
```

这是读取data.cvs中的数据，然后通过这个数据中的信息将数据划分为训练数据和测试数据。

```
tabular_data = pd.read_csv(config.csv)
print(tabular_data)

ds_dict = dataloader.get_split_fold(tabular_data)
patch_size = eval(config.patch_size)
batch_size = config.batch_size
epochs = config.epochs
```

可以看出ds_dict中存放的是数据的路径，并不是实际的数据。

```
ds_dict:
{'train_ds': {'img_npy': ['img_npy/train_1.npy', 'img_npy/train_10.npy', 'img_npy/train_11.npy', 'img_npy/train_12.npy', '
img_npy/train_13.npy', 'img_npy/train_14.npy', 'img_npy/train_15.npy', 'img_npy/train_16.npy', 'img_npy/train_17.npy', 'im
g_npy/train_18.npy', 'img_npy/train_19.npy', 'img_npy/train_2.npy', 'img_npy/train_20.npy', 'img_npy/train_21.npy', 'img_n
py/train_22.npy', 'img_npy/train_23.npy', 'img_npy/train_24.npy', 'img_npy/train_25.npy', 'img_npy/train_26.npy', 'img_npy
/train_27.npy', 'img_npy/train_28.npy', 'img_npy/train_29.npy', 'img_npy/train_3.npy', 'img_npy/train_30.npy', 'img_npy/tr
ain_31.npy', 'img_npy/train_32.npy', 'img_npy/train_33.npy', 'img_npy/train_34.npy', 'img_npy/train_35.npy', 'img_npy/trai
```

`tr_transforms` 是对数据进行增强的函数

```
tr_transforms = dataloader.get_train_transform(patch_size, prob=config.aug_prob)
train_dl = dataloader.DataLoader(data=ds_dict['train_ds'], batch_size=batch_size, patch_size=patch_size,
                                 num_threads_in_multithreaded=4, seed_for_shuffle=5243,
                                 return_incomplete=False, shuffle=True, infinite=True)
train_gen = MultiThreadedAugmenter(train_dl, tr_transforms, num_processes=4,
                                   num_cached_per_queue=2,
                                   seeds=None, pin_memory=False)
val_dl = dataloader.DataLoader(data=ds_dict['testA_ds'], batch_size=batch_size, patch_size=patch_size,
                               num_threads_in_multithreaded=1, seed_for_shuffle=5243,
                               return_incomplete=False, shuffle=True, infinite=True)
```

`def get_train_transform(patch_size, prob=0.5):`

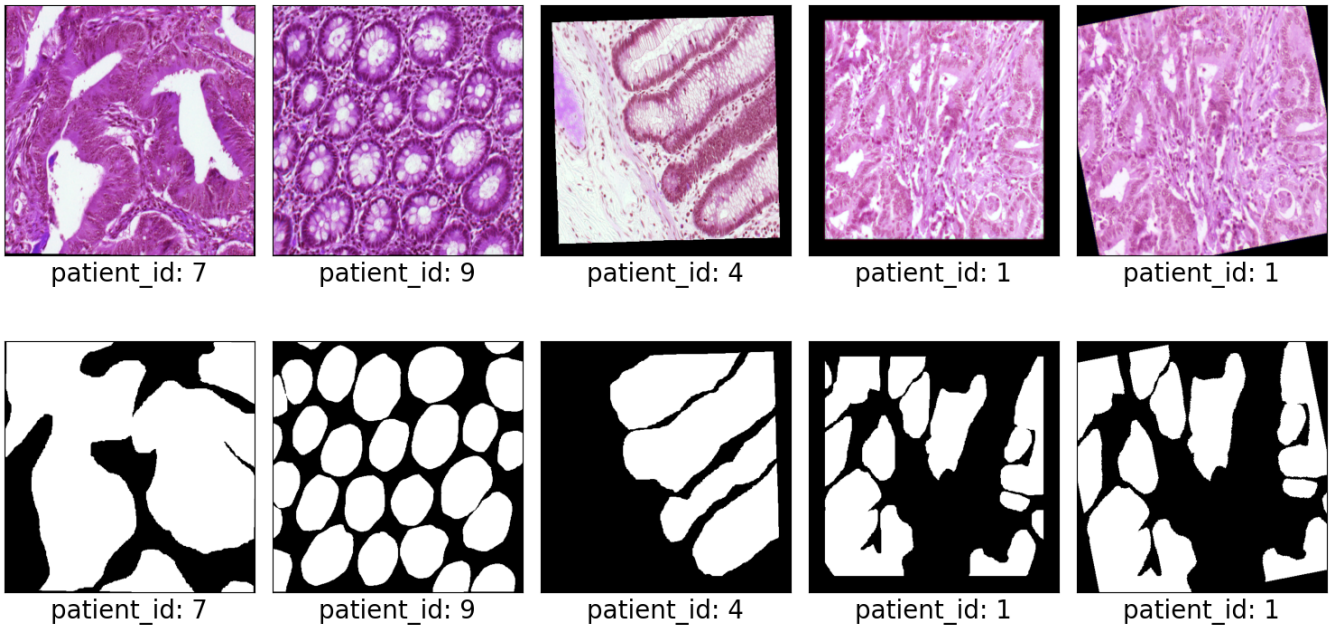
用于获取图像增强的转换列表。在训练神经网络时，数据增强是一种常用的技术，它通过对输入数据进行一

系列变换来增加数据量，增强模型的鲁棒性和泛化能力。`get_train_transform`使用`albumentations`库，

它提供了许多图像增强的方法，如随机裁剪、水平翻转、随机旋转、随机缩放等，可以根据需要自由组合使用。

这里只对训练集合的数据进行了增强：

结果：



可以看出来patient1进行了数据增强，通过旋转操作得到了两张图片。

模型的构建：

```
# define model
model = smp.Unet(encoder_name=config.encoder_model, decoder_use_batchnorm=True,
                 in_channels=3, classes=config.n_class).to(device)
# summary(model, (3, 512, 512))
optimizer = eval(config.optimizer)(model.parameters(), lr=float(config.learning_rate))
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=20,
                                                       factor=0.1)
# xent = nn.BCELoss()
dice_loss = smp.losses.DiceLoss(smp.losses.BINARY_MODE, from_logits=True)
# label smoothing
xent = smp.losses.SoftBCEWithLogitsLoss(smooth_factor=0.1)
```

`train_dl = dataloader.DataLoader:`创建了一个用于训练的数据加载器。

这段代码使用`dataloader`模块中的`DataLoader`类创建了一个用于训练的数据加载器`train_dl`。

其中，`data`参数指定了数据集对象`ds_dict['train_ds']`，`batch_size`指定了每个批次的大小，

`patch_size`指定了每个数据样本的裁剪大小，`num_threads_in_multithreaded`指定了多线程加

载数据时使用的线程数，`seed_for_shuffle`指定了数据集打乱时使用的随机种子，`return_incomplete`

指定了是否返回不完整的批次，`shuffle`指定了是否打乱数据集顺序，`infinite`指定了数据集是否可以无限迭代。这个数据加载器会将数据集中的数据样本按照指定的参数进行处理和组织，并以指定的批次大小作为一个迭代器来供模型训练使用。

`train_gen`是一个用于数据增强的生成器。在训练过程中，我们使用原始图像的随机裁剪、弹性形变、旋转、缩放、翻转等操作来扩充训练数据集。因为原始数据太少，训练出来的结果不好，因此采用了数据增强技术。

`train_gen`的作用是从训练数据集中不断读取批次的图像，并在读取每个批次图像时应用指定的增强操作。

网络的编码器部分使用的预训练模型的名称或类型。这里使用的是`resnet18`网络。

图像的通道数是RGB三通道，类的数量是1。

这里优化器选择的是`optim.Adam`。

optim.Adam是一个优化算法，用于训练神经网络。它是一种基于梯度下降的优化算法，能够自适

应地调整学习率，并可以更好地处理稀疏梯度和噪声数据。Adam算法通过计算梯度的一阶和二阶矩估计来更新每个参数的学习率

`smp.losses.DiceLoss` 是一种用于图像分割任务的损失函数。

`smp.losses.SoftBCEWithLogitsLoss` 是一种用于二分类问题的损失函数。 它将sigmoid函数应用于网络的输出以将其转换为概率值，并计算二分类交叉熵损失。 这种损失函数可以通过添加一个

平滑系数来减轻过拟合的风险。

训练函数：

这个训练函数，通过传递一个模型和一个优化器类型来进行训练。

返回值是一个损失值。

通过batch_size和数据的数量计算出每个epoch的batch_num。

训练中每次挑选从数据集中挑选batch_size数量的数据，然后进行训练，得到结果的损失值。

通过选择的优化器，这里选择的是梯度下降法，然后更新网络的权重，向后传递。

当所有批次训练完成后，求所有批次训练结果的平均损失函数作为最终损失。

```
def train(model, optimizer):
    # total number of training batches
    num_batches = math.ceil(len(ds_dict['train_ds']['img_npy'])/
batch_size)
    model.train()
    batch_xent_l = []
    batch_dice_l = []
    batch_loss = []
    print("Training...")
    for i in tqdm(range(num_batches)):
        train_batch = next(train_gen)
        imgs = train_batch['data']
        segs = train_batch['seg']
        # normalization
        imgs = utils.min_max_norm(imgs)
        # binarisation
        segs = np.where(segs > 0., 1.0, 0.).astype('float32')
        segs = np.expand_dims(segs[:, 0, :, :], 1)
```

```
        imgs, segs = torch.from_numpy(imgs).to(device), torch.from_numpy(segs).to(device)
        # Compute loss
        pred = model(imgs),
        loss, xent_l, dice_l = custom_loss(pred, segs)
        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        # batch losses
        batch_xent_l.append(xent_l)
        batch_dice_l.append(dice_l)
        batch_loss.append(loss)
    # apply sigmoid to masking
    segs = nn.Sigmoid()(segs)
    # taking the average along the batch
    loss = torch.mean(torch.as_tensor(batch_loss)).item()
    avg_xent_l = torch.mean(torch.as_tensor(batch_xent_l)).item()
    avg_dice_l = torch.mean(torch.as_tensor(batch_dice_l)).item()

    return {'loss': loss, 'xent_l': avg_xent_l, 'dice_l': avg_dice_l,
            'imgs': imgs.cpu().detach().numpy(),
            'segs': segs.cpu().detach().numpy(),
            'pred': pred.cpu().detach().numpy()}
```

测试函数：

这是测试函数，基本和训练的步骤一致，只是这里不用优化器了，不需要对参数进行先后传播的过程。

只需要计算出损失

值就可以了。

```
def test(model):
    num_batches = math.ceil(len(ds_dict['testA_ds']['img_npy']) / batch_size)
    model.eval()
    # no need back prop for testing set
```

```
    batch_xent_l = []
    batch_dice_l = []
    batch_loss = []
    print("Testing...")
    with torch.no_grad():
        for i in tqdm(range(num_batches)):
            val_batch = next(val_dl)
            imgs = val_batch['data']
            segs = val_batch['seg']
            # normalization
            imgs = utils.min_max_norm(imgs)
            # binarisation
            segs = np.where(segs > 0., 1.0, 0.).astype('float32'
)
            segs = np.expand_dims(segs[:, 0, :, :], 1)
            imgs, segs = torch.from_numpy(imgs).to(device), torc
h.from_numpy(segs).to(device)
            # Compute loss
            pred = model(imgs)
            loss, xent_l, dice_l = custom_loss(pred, segs)
            # batch losses
            batch_xent_l.append(xent_l)
            batch_dice_l.append(dice_l)
            batch_loss.append(loss)
        # apply sigmoid to masking
        segs = nn.Sigmoid()(segs)
        # taking the average along the batch
        loss = torch.mean(torch.as_tensor(batch_loss)).item()
        avg_xent_l = torch.mean(torch.as_tensor(batch_xent_l)).i
tem()
        avg_dice_l = torch.mean(torch.as_tensor(batch_dice_l)).i
tem()
    return {'loss': loss, 'xent_l': avg_xent_l, 'dice_l': avg_di
ce_l,
            'imgs': imgs.cpu().detach().numpy(),
            'segs': segs.cpu().detach().numpy(),
            'pred': pred.cpu().detach().numpy()}

```

总体模型训练过程：

主函数每一个epoch中使用train函数和test函数一次。

然后展示了每一次训练过程中训练集和测试集中损失和得分情况。

再把这些结果保存到wandb中，通过这个将损失情况进行可视化。

训练过程中会通过scheduler.step(test_output['loss'])动态的调整学习率。

在最后会检查这次训练的结果是否更优，如果更优的话，会将模型的参数重新保存起来。

```
def main():
    current_total_loss = 1000
    current_dice_score = 0
    for e in range(1, epochs+2):
        print("Epcohs:", e)
        train_output = train(model, optimizer)
        test_output = test(model)
        scheduler.step(test_output['loss'])
        print("Training Outputs: ")
        print("Total loss: {:.2f}, BCE: {:.2f}, Dice Score: {:.2f}".format(train_output['loss'], train_output['xent_l'], 1 - train_output['dice_l']))
        print("-"*100)
        print("Validation Outputs: ")
        print("Total loss: {:.2f}, BCE: {:.2f}, Dice Score: {:.2f}".format(test_output['loss'], test_output['xent_l'], 1-test_output['dice_l']))
        # logging
        wandb.log({"Train_total_loss": train_output['loss'], "Val_total_loss": test_output['loss']}, step=e)
        wandb.log({"Train_BCE_loss": train_output['xent_l'], "Val_BCE_loss": test_output['xent_l']}, step=e)
        wandb.log({"Train_dice_score": 1-train_output['dice_l'], "Val_dice_score": 1-test_output['dice_l']}, step=e)
        wandb.log({"Learning rate": optimizer.param_groups[0]["lr"]}, step=e)
        if e%10==0:
            # threshold sigmoid output with 0.5
            pred_thr = np.where(test_output['pred']>0.5, 1.0, 0.0)
            # sample a dataset from the batch for visualization purpose
            imgs = [test_output['imgs'][0, 0, :, :], test_output['segs'][0, 0, :, :], pred_thr[0, 0, :, :]]
            captions = ['Gland Image', 'Masking', 'Prediction']
            fig = utils.plot_comparison(imgs, captions, plot=Fal
```

```
se, n_col=len(imgs),
                                    figsize=(12, 12), cmap='gray')
            wandb.log({"Validation Dataset Output Sample": wand
b.Image(fig)}, step=e)

        # save model
        # weights_dir = '/home/kevinteng/Desktop/weights/'
        # weights_dir = 'weights/'
        weights_dir = 'myresult/'
        print(os.path.abspath(weights_dir))
        if not os.path.exists(weights_dir):
            os.makedirs(weights_dir)
        base_path = os.path.split(weights_dir)[0]
        if test_output['loss'] < current_total_loss:
            current_total_loss = test_output['loss']
            torch.save(model.state_dict(), weights_dir+'best_los
s_{}.pth'.format(e))
            wandb.save(os.path.join(weights_dir, 'best_loss_{}.p
th'.format(e)), base_path=base_path)
        if (1-test_output['dice_l']) > current_dice_score:
            current_dice_score = 1-test_output['dice_l']
            torch.save(model.state_dict(), weights_dir+'best_dic
e_{}.pth'.format(e))
            wandb.save(os.path.join(weights_dir, 'best_dice_{}.p
th'.format(e)), base_path=base_path)
        print()

    print("Model training runtime: {} mins".format((time.time()
 - start)/60.0))
```

构建完成的模型展示：这里显示了模型参数的总数，以及各个层直接的参数数量。

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1         [-1, 64, 256, 256]           9,408
       BatchNorm2d-2         [-1, 64, 256, 256]             128
```

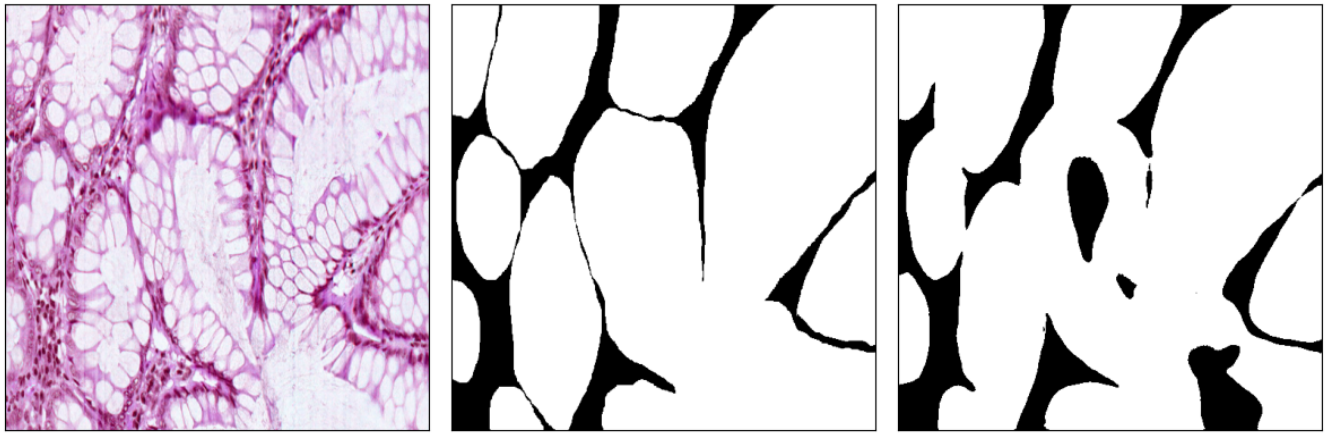
```
            Conv2d-116         [-1, 16, 512, 512]           2,304
       BatchNorm2d-117         [-1, 16, 512, 512]              32
              ReLU-118         [-1, 16, 512, 512]               0
          Identity-119         [-1, 16, 512, 512]               0
         Attention-120         [-1, 16, 512, 512]               0
      DecoderBlock-121         [-1, 16, 512, 512]               0
       UnetDecoder-122         [-1, 16, 512, 512]               0
            Conv2d-123          [-1, 1, 512, 512]             145
          Identity-124          [-1, 1, 512, 512]               0
          Identity-125          [-1, 1, 512, 512]               0
        Activation-126          [-1, 1, 512, 512]               0
================================================================
Total params: 14,328,209
Trainable params: 14,328,209
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 3.00
Forward/backward pass size (MB): 1139.00
Params size (MB): 54.66
Estimated Total Size (MB): 1196.66
----------------------------------------------------------------
```

载入模型，测试结果：

```
    model.load_state_dict(torch.load(weight_path))
    start = time.time()
    test_output = test(model)
    print("Runtime: {} s".format(time.time()-start))
    print("Total loss: {:.2f}, BCE: {:.2f}, Dice Loss: {:2f}".format(test_output['loss'],
        test_output['xent_l'], test_output['dice_l']))
```

```
Testing...
100%|██████████| 4/4 [00:05<00:00,  1.29s/it]
Runtime: 5.161027193069458 s
Total loss: 0.59, BCE: 0.47, Dice Loss: 0.121558
```

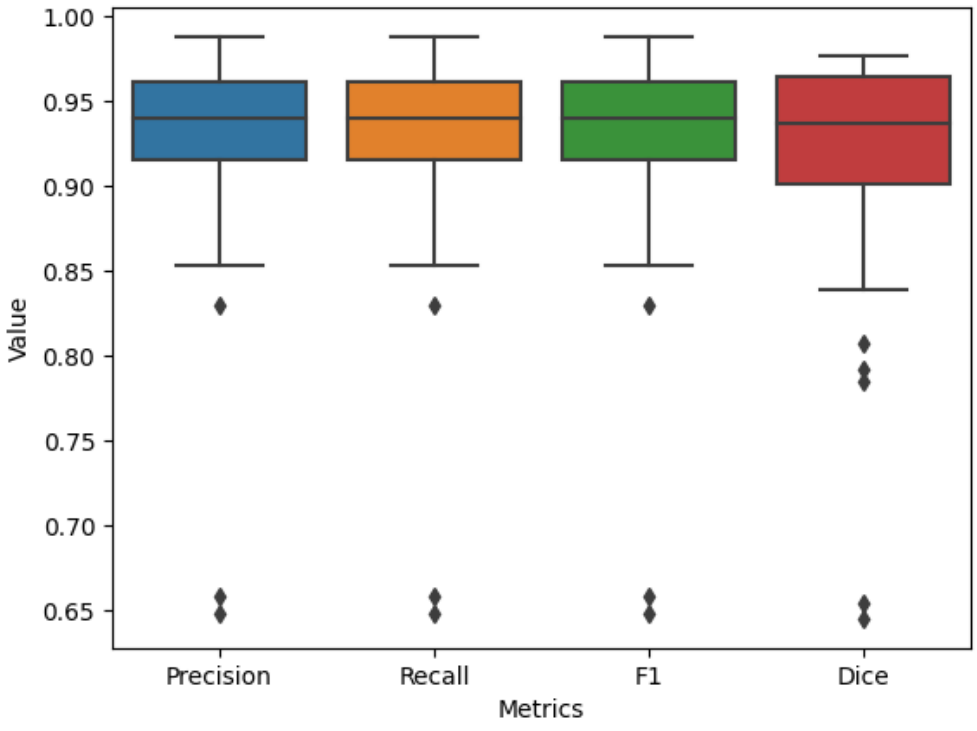
原始图像标注的语义分割图片，预测的语义分割图片对比。



在数据集A中的得分情况。

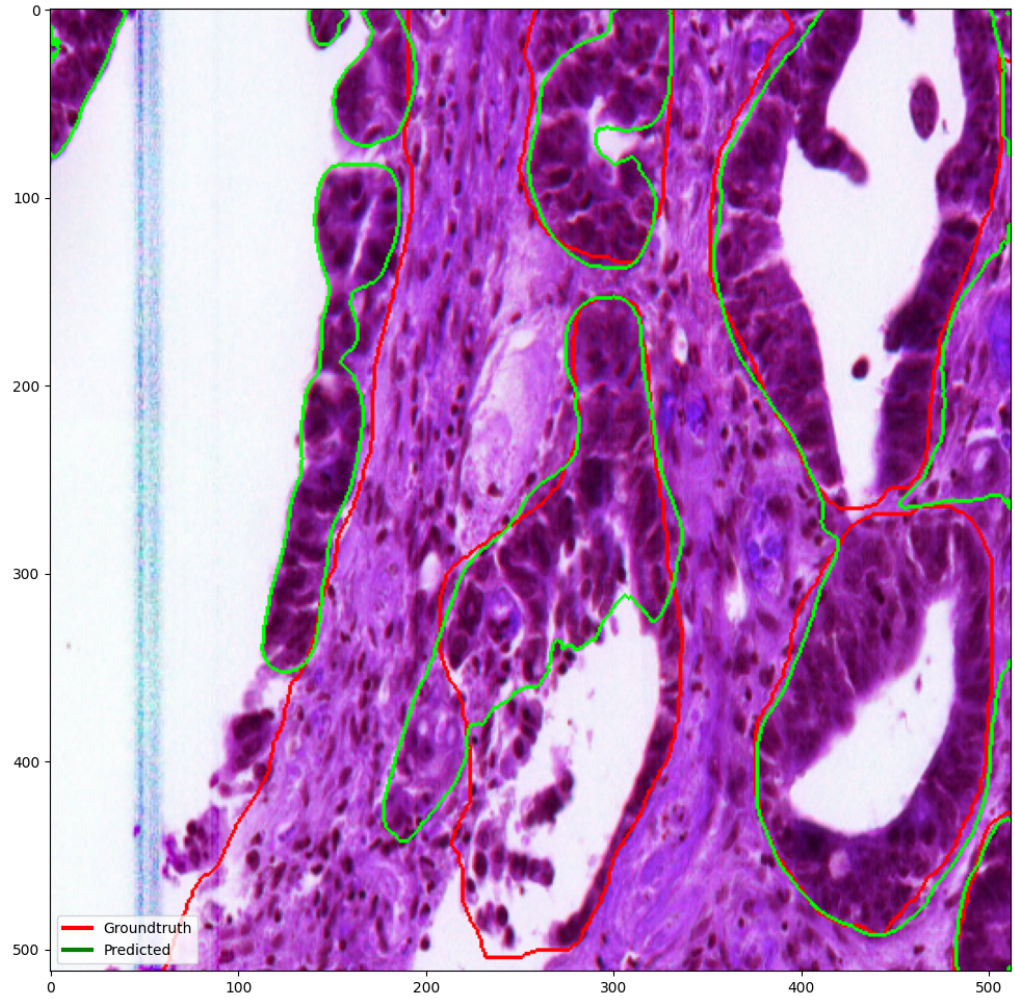
```
Dataset: testA_ds
--------------------
Precision: 0.93 (0.06)
Recall: 0.93 (0.06)
F1-score: 0.93 (0.06)
Dice score: 0.92 (0.07)
Hausdorff Distance: 83.35 (66.77)
```

得分情况的箱线图：



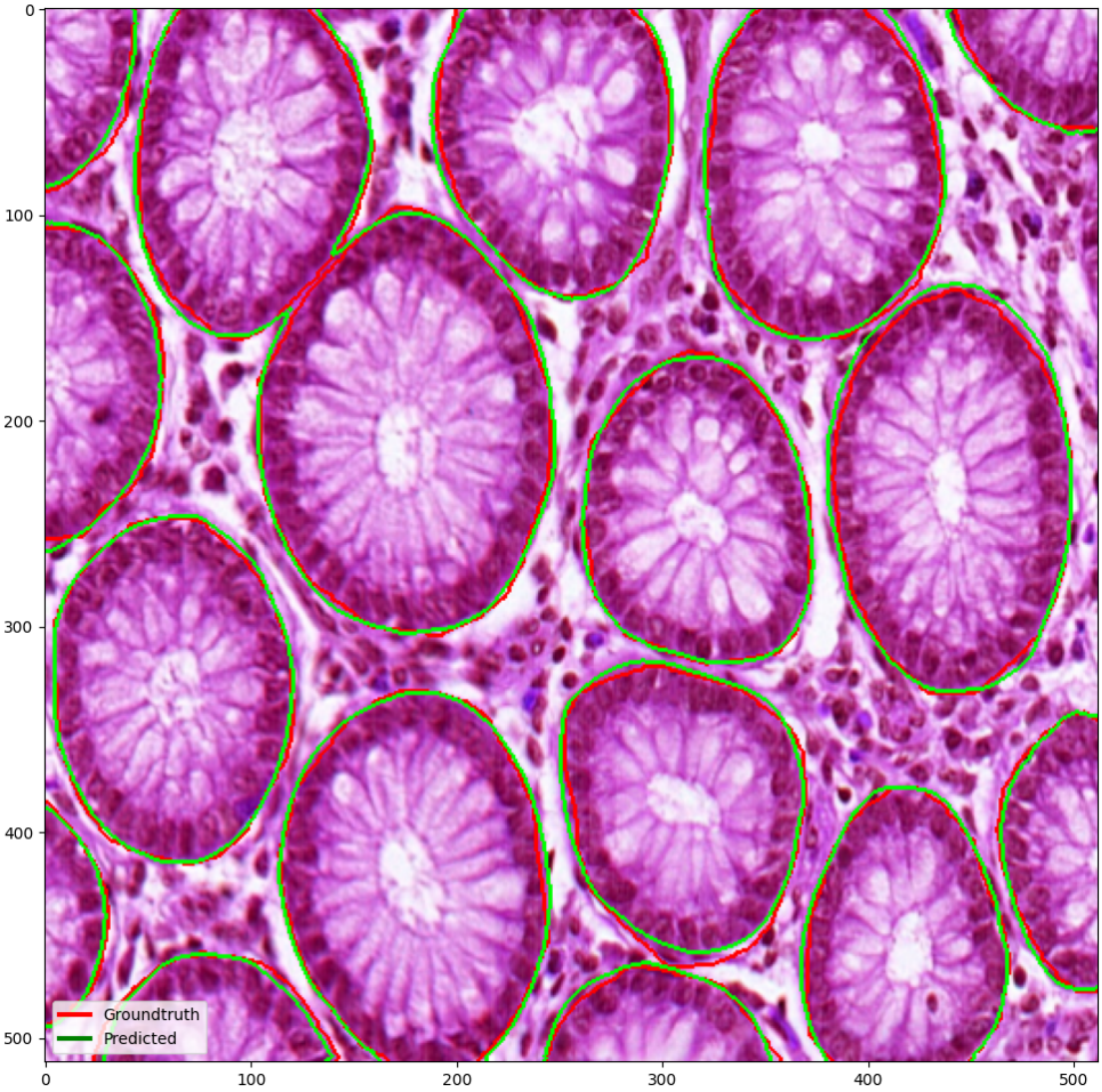
最低Dice得分下的分割情况：

预测的结果和标注值在原始图片中展示出来的结果。可以看出预测和实际的结果只有一小部分有些误差。



最高Dice得分下的分割情况：

几乎和标注信息完美重合。



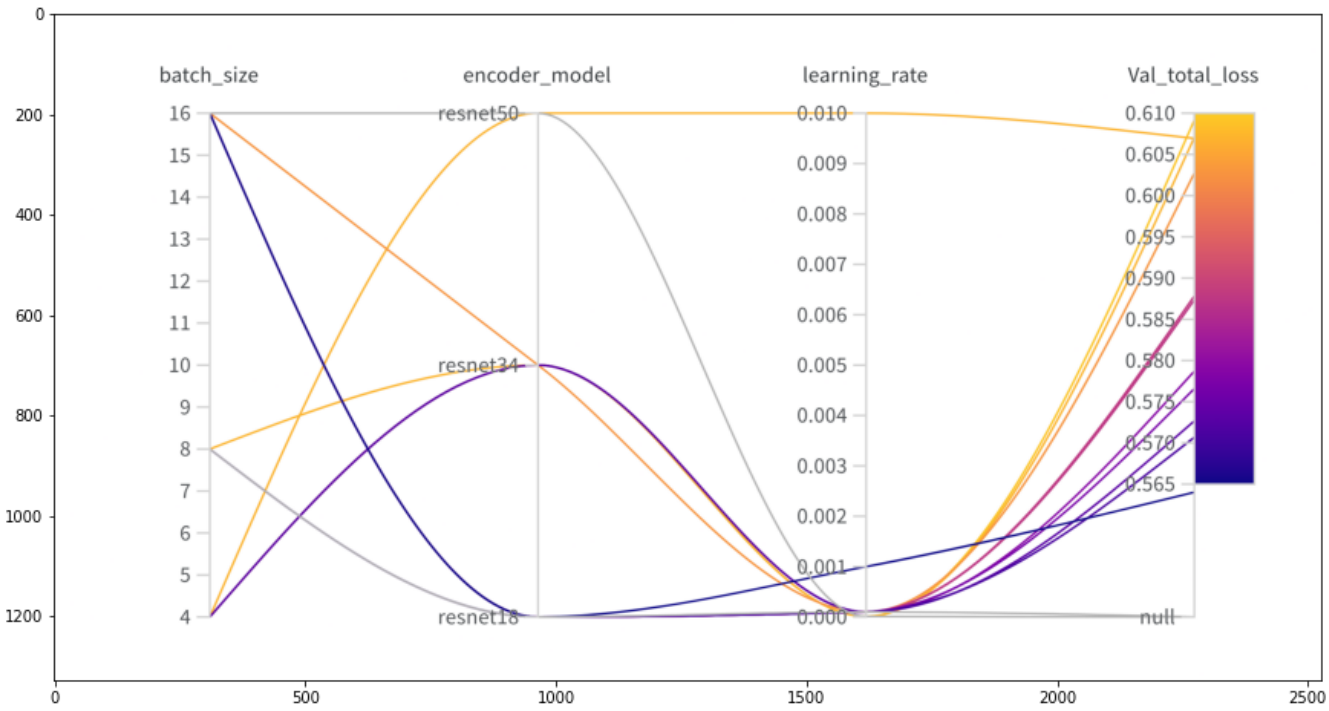
改变超参数，定义不同的学习率，batchsize的大小，以及使用不同的编码器进行对比实验。

Hyperparameter tuning with wandb sweep techniques, using grid search. The hyperparameters tuned are as below:

- batch size: [4, 8, 16]
- learning rate: [1e-2, 1e-3, le-4, 1e-5]
- encoder model: [resnet18, resnet34, resnet50]

```
plt.figure(figsize=(15, 8))
plt.imshow(np.array(plt.imread('fig/hyperparam_tuning.png')))
```

对比结果：



最好的超参数选取是：

Best hyperparameter:

- batch size: 16
- encoder model: resnet 18
- learning rate: 0.001

三、 总结（心得体会）

在这个实验中，了解了图像分割技术和目标检测技术和分类技术的区别。分割中的神经网络没有全连接层，不用将卷积提取的特征展开成一个向量，然后通过全连接层去进行分类。

他需要输出一个和原始图像相同大小的图片，在通过卷积网络提取出图片的特征后需要进行上采样操作，从特征中恢复成原始图片大小的过程。